# 1. Software Test Guide

**DICTIONARY_STDLIB**

| Document Control | |
|---|---|
| Author(s) | Michael Gardner |
| Version | 0.1.0 |
| Status | Draft |
| Status Date | 2026-04-07 |
| SPDX-License-Identifier | BSD-3-Clause |
| License File | See the LICENSE file in the project root |
| Copyright | © 2026 Michael Gardner, A Bit of Help, Inc. |

| Project Profile | |
|---|---|
| Variant | application |
| Library Role | N/A |
| Application Role | service |
| Assurance | non-spark |
| Execution | concurrent |
| Parallelism | bounded |
| Platform | server |
| Execution Environment | linux |
| Processor Architecture | amd64 |
| Deployment | containerized |

# Table of Contents


1. Software Test Guide ..... 1
2. Introduction ..... 1
   - 2.1. Purpose ..... 1
   - 2.2. Scope ..... 1
   - 2.3. References ..... 1
3. Test Strategy ..... 1
   - 3.1. Test Categories ..... 1
   - 3.2. Testing Philosophy ..... 1
4. Test Organization ..... 1
   - 4.1. Directory Structure ..... 1
   - 4.2. Naming Conventions ..... 2
   - 4.3. Build Configuration ..... 2
5. Test Framework ..... 2
   - 5.1. Framework Overview ..... 2
   - 5.2. Framework API ..... 2
   - 5.3. Usage Pattern ..... 3
6. Test Execution ..... 3
   - 6.1. Running All Tests ..... 3
   - 6.2. Running Specific Suites ..... 3
   - 6.3. Expected Output ..... 3
   - 6.4. Docker Tests ..... 3
7. Test Details ..... 4
   - 7.1. Unit Tests ..... 4
   - 7.2. Integration Tests ..... 4
   - 7.3. End-to-End Tests ..... 4
8. Writing New Tests ..... 4
   - 8.1. Test Template ..... 4
   - 8.2. Adding to Build Configuration ..... 4
   - 8.3. Test Doubles ..... 5
9. Traceability ..... 5
   - 9.1. Requirements to Tests ..... 5
   - 9.2. Test-Level Coverage ..... 5
10. Test Maintenance ..... 5
    - 10.1. When to Update ..... 5
    - 10.2. Quality Guidelines ..... 5
    - 10.3. Continuous Integration ..... 5
11. Appendices ..... 6
    - 11.1. Change History ..... 6
12. Appendices ..... 7
    - 12.1. Change History ..... 7

# 2. Introduction

## 2.1. Purpose

This Software Test Guide (STG) describes the testing strategy, test organization, execution procedures, and traceability for **Dictionary_Stdlib**, an educational REST microservice built exclusively with Ada 2022 standard library packages.

## 2.2. Scope

This document covers:

- unit, integration, and end-to-end testing strategy,
- test framework design and usage patterns,
- test execution via Make targets,
- traceability between SRS requirements and test suites, and
- guidance for writing new tests.

## 2.3. References

- Software Requirements Specification (SRS) for Dictionary_Stdlib.
- Software Design Specification (SDS) for Dictionary_Stdlib.

# 3. Test Strategy

## 3.1. Test Categories

| Category | Location | Count | Purpose |
|---|---|---|---|
| Unit | `test/unit/` | 99 | Verify individual packages in isolation. |
| Integration | `test/integration/` | 26 | Verify the router pipeline (request to response). |
| End-to-End | `test/e2e/` | 28 | Verify all endpoints via HTTP against a running server. |

Total: 152 tests.

## 3.2. Testing Philosophy

- Each package is tested through its public API. No white-box access to private state.
- The store's protected object is tested through its public operations, not by inspecting the internal map.
- Integration tests construct HTTP Request records directly and pass them to the router, bypassing sockets. This isolates the pipeline from network timing.
- E2E tests use curl against a live server to validate the full stack including TCP, HTTP parsing, and response formatting.
- Error paths are tested as thoroughly as success paths: invalid keys, duplicate entries, missing entries, malformed JSON, wrong HTTP methods, unknown paths.

# 4. Test Organization

## 4.1. Directory Structure

```
test/
├── common/
│   ├── test_framework.ads       Shared assert/report framework
│   └── test_framework.adb
├── unit/
│   ├── unit_tests.gpr           GPR project for unit tests
```

```
│   └── src/
│       ├── unit_runner.adb      Main: calls all unit test packages
│       ├── test_bounded_text.adb
│       ├── test_http.adb
│       ├── test_json.adb
│       ├── test_store.adb
│       └── test_validation.adb
├── integration/
│   ├── integration_tests.gpr    GPR project for integration tests
│   └── src/
│       ├── integration_runner.adb
│       └── test_router.adb
├── e2e/
│   └── e2e_test.sh              Shell script: curl-based tests
└── bin/                         Test executables (built here)
```

### 4.2. Naming Conventions

| Element | Convention | Example |
|---|---|---|
| Runner | `<category>_runner.adb` | `unit_runner.adb` |
| Test file | `test_<package>.adb` | `test_validation.adb` |
| Test name | Descriptive string in Assert call | `"key: case-insensitive equality"` |

### 4.3. Build Configuration

Each test suite has its own GPR project that `withs` the main `dictionary_stdlib.gpr`. This gives test code access to all Dictionary packages without duplicating source paths.

- `test/unit/unit_tests.gpr` builds `test/bin/unit_runner`
- `test/integration/integration_tests.gpr` builds `test/bin/integration_runner`
- E2E tests require no compilation (shell script).

## 5. Test Framework

### 5.1. Framework Overview

`Test_Framework` is a minimal custom package providing assertion, result tracking, and summary reporting. It has no external dependencies.

### 5.2. Framework API

```
package Test_Framework is
   procedure Assert
     (Condition : Boolean;
      Test_Name : String;
      Message   : String := "");
   procedure Report;
   function Pass_Count return Natural;
   function Fail_Count return Natural;
   function All_Passed return Boolean;
end Test_Framework;
```

- `Assert` increments the pass or fail counter. On failure, it prints the test name and optional message.
- `Report` prints a summary (total, pass, fail, result).
- `All_Passed` returns True when `Fail_Count = 0` and at least one test ran.
- The runner sets exit code 0 (pass) or 1 (fail) based on `All_Passed`.

### 5.3. Usage Pattern

```
with Test_Framework;
with Dictionary.Validation;

package body Test_Validation is
   procedure Run is
   begin
      Test_Framework.Assert
        (Dictionary.Validation.Is_Valid_Key ("hello"),
         "valid_key: hello");
      Test_Framework.Assert
        (not Dictionary.Validation.Is_Valid_Key (""),
         "valid_key: reject empty");
   end Run;
end Test_Validation;
```

## 6. Test Execution

### 6.1. Running All Tests

```
make test-all      # Unit + integration (compiled suites)
make test-e2e      # E2E (starts server, runs curl, stops server)
```

### 6.2. Running Specific Suites

```
make test-unit          # 99 unit tests
make test-integration   # 26 integration tests
make test-e2e           # 28 E2E tests
```

### 6.3. Expected Output

Successful run:

```
Dictionary Unit Tests
====================
--- Validation ---
--- Bounded_Text ---
--- JSON ---
--- HTTP ---
--- Store ---
========================================
  Total:  99
  Pass:   99
  Fail:   0
========================================
  Result: ALL PASSED
========================================
```

Failed tests print `FAIL: <test_name>` with an optional diagnostic message before the summary.

### 6.4. Docker Tests

```
make docker-test   # Build image, start container, run curl tests, stop
```

# 7. Test Details

## 7.1. Unit Tests

| Suite | Count | Covers |
|---|---|---|
| `test_bounded_text` | 14 | Create, To_String, Length, Empty, case normalization, ordering, equality for both Key_Text and Value_Text. |
| `test_http` | 16 | Method_From_String, status code/reason helpers, Parse_Request for GET/POST/DELETE, Format_Response, malformed input. |
| `test_json` | 11 | Parse_Entry (valid, value-only, malformed, empty), Serialize_Entry, Serialize_Entry_List (empty, populated), Serialize_Error, Serialize_Health. |
| `test_store` | 28 | Create, Get, Get_All (sorted), Update, Delete, Contains, Count, duplicate detection, case-insensitive duplicates, not-found cases, store operations sequencing. |
| `test_validation` | 30 | Is_Key_Char (valid/invalid chars), Is_Value_Char (valid/invalid), Is_Valid_Key (good/bad/boundary lengths), Is_Valid_Value (good/bad/boundary lengths). |

## 7.2. Integration Tests

`test_router` (26 tests) exercises the full internal pipeline by constructing HTTP Request records and passing them to `Dictionary.Router.Handle_Request`:

- Health endpoint: GET 200, POST 405.
- CRUD lifecycle: create 201, duplicate 409, case-insensitive duplicate 409, get 200, get missing 404, list sorted 200, update 200, update key mismatch 400, update missing 404, delete 204, delete again 404.
- Validation errors: bad key 400, empty JSON 400, malformed JSON 400.
- Method not allowed: DELETE /entries 405, POST /entries/{key} 405.
- Unknown path: 404.

## 7.3. End-to-End Tests

`e2e_test.sh` (28 tests) starts the server binary, exercises all endpoints via curl, verifies HTTP status codes and response body content, then stops the server:

- Health, create (3 entries), duplicate, case-insensitive duplicate, get one, get missing, list sorted, update, verify update, update missing, update key mismatch, delete, delete again, bad key, bad JSON, method not allowed (2), unknown path.

# 8. Writing New Tests

## 8.1. Test Template

1. Create `test/unit/src/test_<package>.ads` (spec with `procedure Run;`).
2. Create `test/unit/src/test_<package>.adb` (body with assertions).
3. Add `with Test_<Package>;` and `Test_<Package>.Run;` call to `unit_runner.adb`.
4. Build: `make build-tests`.

## 8.2. Adding to Build Configuration

Test source files in `test/unit/src/` or `test/integration/src/` are automatically included by the corresponding GPR project. Only the runner needs a new `with` and `Run` call.

### 8.3. Test Doubles

This project does not use mocks or test doubles. The store is the only stateful component, and it is tested directly through its protected interface. Integration tests exercise the real store, router, and JSON packages together.

## 9. Traceability

### 9.1. Requirements to Tests

| Requirement | Suite | Test Coverage |
|---|---|---|
| FR-01.1 – FR-01.6 | Unit | `test_validation.adb`, `test_bounded_text.adb` |
| FR-02.1 – FR-02.6 | Unit | `test_store.adb` |
| FR-03.1 – FR-03.6 | Unit | `test_http.adb` |
| FR-04.1 – FR-04.4 | Integration | `test_router.adb` |
| FR-05.1 – FR-05.5 | E2E | `e2e_test.sh` |
| FR-06.1 – FR-06.2 | E2E | `e2e_test.sh` |
| NFR-06.1 – NFR-06.4 | Docker | `make docker-test` |

### 9.2. Test-Level Coverage

| Test Level | Coverage Summary |
|---|---|
| Unit | All public functions in Validation, Bounded_Text, JSON, HTTP, and Store packages. |
| Integration | Full router pipeline: request parsing through response generation, all routes and error paths. |
| E2E | All HTTP endpoints exercised via curl, including success paths, error paths, and edge cases. |
| Docker | Production image build, nonroot execution, health check, endpoint functionality. |

## 10. Test Maintenance

### 10.1. When to Update

- Add tests when new endpoints or validation rules are introduced.
- Add regression tests when defects are fixed.
- Update E2E tests when the API contract changes.
- Update integration tests when routing logic changes.

### 10.2. Quality Guidelines

- Test names should describe the behavior being verified, not the implementation.
- Tests should not depend on execution order (unit and integration suites share a single store instance – be aware of state accumulation in store tests).
- Failure messages should be specific enough to diagnose without reading the test source.

### 10.3. Continuous Integration

Tests are executed via Make targets. A CI pipeline should run:

1. `make build` – verify clean compilation.
2. `make test-unit` – 99 unit tests.
3. `make test-integration` – 26 integration tests.
4. `make test-e2e` – 28 E2E tests.

5. `make docker-test` – build and validate the production image.

# 11. Appendices

## 11.1. Change History

| Version | Date | Author | Changes |
|---|---|---|---|
| 0.1.0 | 2026-04-07 | Michael Gardner | Initial STG for the dictionary_stdlib educational microservice. |

# 12. Appendices

## 12.1. Change History

| Version | Date | Author | Changes |
|---|---|---|---|
| 0.1.0 | 2026-04-07 | Michael Gardner | Initial STG for the dictionary_stdlib educational microservice. |